

```
const { Client, GatewayIntentBits, Partial, REST, Routes, SlashCommandBuilder, A
const { GoogleGenerativeAI } = require('@google/generative-ai');
```

```
// — CONFIG —
```

```
const DISCORD_TOKEN = process.env.DISCORD_TOKEN;
const GEMINI_API_KEY = process.env.GEMINI_API_KEY;
const CLIENT_ID = process.env.CLIENT_ID;
const GUILD_ID = process.env.GUILD_ID;
```

```
// — OEUVRES —
```

```
const OEUVRES = {
  'Douai': {
    label: 'Douai',
    nom: 'CAHIERS DE DOUAI — Arthur Rimbaud',
    description: 'Recueil de poèmes de jeunesse de Rimbaud, écrit en 1870.'
  },
  'DSV': {
    label: 'DSV',
    nom: 'DISCOURS SUR LA VIOLENCE — textes variés',
    description: 'Groupement de textes autour du discours et de la violence.'
  },
  'LeMenteur': {
    label: 'Le Menteur',
    nom: 'LE MENTEUR — Corneille',
    description: 'Comédie de Corneille (1644), pièce fondatrice du genre.'
  },
  'ExpressionRage': {
    label: 'Expression Rage',
    nom: "PONGE — L'EXPRESSION DE LA RAGE",
    description: "Textes de Francis Ponge autour de l'expression poétique de la r
  },
  'PeauDeChagrin': {
    label: 'Peau de Chagrin',
    nom: 'LA PEAU DE CHAGRIN — Balzac',
    description: 'Roman philosophique de Balzac (1831).'
  }
};
```

```
// — ÉTAT SESSIONS —
```

```
const qcmSessions = new Map(); // userId → session QCM
const essaySessions = new Map(); // threadId → session essay
```

```
// — INIT GEMINI —
```

```
const genAI = new GoogleGenerativeAI(GEMINI_API_KEY);
```

```

const model = genAI.getGenerativeModel({
  model: 'gemini-1.5-flash',
  generationConfig: { temperature: 0.7, maxOutputTokens: 4096 }
});

async function callGemini(prompt) {
  const result = await model.generateContent(prompt);
  return result.response.text();
}

// — CLIENT DISCORD —————
const client = new Client({
  intents: [
    GatewayIntentBits.Guilds,
    GatewayIntentBits.GuildMessages,
    GatewayIntentBits.MessageContent,
    GatewayIntentBits.GuildMembers,
    GatewayIntentBits.DirectMessages,
  ],
  partials: [Partials.Channel, Partials.Message]
});

// — REGISTER SLASH COMMANDS —————
async function registerCommands() {
  const commands = [
    new SlashCommandBuilder()
      .setName('qcm')
      .setDescription('Lance un QCM interactif sur une œuvre au programme du Bac'),
    new SlashCommandBuilder()
      .setName('essay')
      .setDescription('Génère une problématique et t\'accompagne dans la rédaction'),
    new SlashCommandBuilder()
      .setName('end')
      .setDescription('Termine ta dissertation et reçois une correction détaillée'),
    new SlashCommandBuilder()
      .setName('close')
      .setDescription('Ferme ce thread'),
  ].map(cmd => cmd.toJSON());

  const rest = new REST({ version: '10' }).setToken(DISCORD_TOKEN);
  await rest.put(Routes.applicationGuildCommands(CLIENT_ID, GUILD_ID), { body: commands });
  console.log('✅ Slash commands enregistrées.');
```

```

// — LIRE LE CONTENU D'UNE CATÉGORIE —————
async function lireContenuCategorie(guild, oeuvreKey) {
  const oeuvre = OEUVRES[oeuvreKey];
```

```

const nomOeuvre = oeuvre.nom.toLowerCase();

// Cherche la catégorie correspondante
const categories = guild.channels.cache.filter(c =>
  c.type === 4 && // CategoryChannel
  (c.name.toLowerCase().includes(nomOeuvre.split('-')[0].trim().toLowerCase())
  c.name.toLowerCase().includes(oeuvreKey.toLowerCase())))
);

let contenu = `Œuvre : ${oeuvre.nom}\n\n`;

if (categories.size === 0) {
  contenu += `[Aucune catégorie trouvée pour cette œuvre. Utilise les connaissances]`;
  return contenu;
}

const categorie = categories.first();
const salons = guild.channels.cache.filter(c =>
  c.parentId === categorie.id && c.isTextBased()
);

for (const [, salon] of salons) {
  contenu += `\n--- Salon : #${salon.name} ---\n`;
  try {
    const messages = await salon.messages.fetch({ limit: 100 });
    const sorted = [...messages.values()].reverse();
    for (const msg of sorted) {
      if (msg.content) contenu += `[Message] ${msg.content}\n`;
      if (msg.attachments.size > 0) {
        msg.attachments.forEach(a => {
          contenu += `[Pièce jointe] ${a.name} : ${a.url}\n`;
        });
      }
      if (msg.embeds.length > 0) {
        msg.embeds.forEach(e => {
          if (e.title) contenu += `[Embed titre] ${e.title}\n`;
          if (e.description) contenu += `[Embed description] ${e.description}\n`;
        });
      }
    }
  } catch (e) {
    contenu += `[Impossible de lire ce salon]\n`;
  }
}

return contenu;
}

```

```

// ——— COMMANDE /qcm ———
async function handleQcm(interaction) {
  await interaction.reply({
    content: '📖 **Bac de Français 2026** – Combien de questions veux-tu ? (entre  
ephemeral: true
  });

  const filter = m => m.author.id === interaction.user.id && !isNaN(m.content) &&
  const channel = interaction.channel;

  let collected;
  try {
    collected = await channel.awaitMessages({ filter, max: 1, time: 30000, errors
  } catch {
    return interaction.followUp({ content: '⌚ Temps écoulé. Relance `/qcm` pour
  }

  const nbQuestions = parseInt(collected.first().content);
  await collected.first().delete().catch(() => {});

  // Sélection de l'œuvre
  const selectMenu = new StringSelectMenuBuilder()
    .setCustomId(`qcm_oeuvre_${interaction.user.id}`)
    .setPlaceholder('Choisis une œuvre')
    .addOptions(
      { label: 'Douai', description: 'Cahiers de Douai – Rimbaud', value: 'Douai'
      { label: 'DSV', description: 'Discours S.V.', value: 'DSV' },
      { label: 'Le menteur', description: 'Le menteur – Corneille', value: 'LeMen
      { label: 'Expression Rage', description: 'Ponge – L'Expression de la Rage",
      { label: 'Peau de Chagrin', description: 'La Peau de Chagrin – Balzac', val
    );

  const row = new ActionRowBuilder().addComponents(selectMenu);

  await interaction.followUp({
    content: `✅ **${nbQuestions} question(s)** – Choisis maintenant l'œuvre :`,
    components: [row],
    ephemeral: true
  });

  qcmSessions.set(interaction.user.id, { nbQuestions, step: 'awaiting_oeuvre' });
}

// ——— LANCEMENT QCM APRÈS SÉLECTION ———
async function lancerQcm(interaction, oeuvreKey, nbQuestions) {
  await interaction.deferUpdate();

```

```

const oeuvre = OEUVRES[oeuvreKey];
const contenu = await lireContenuCategorie(interaction.guild, oeuvreKey);

const prompt = `Tu es un professeur de Français pour le Bac 2026 en France.
En te basant sur les informations suivantes sur l'œuvre "${oeuvre.nom}" :

${contenu}

Génère exactement ${nbQuestions} question(s) QCM en français sur cette œuvre.
Chaque question doit avoir exactement 4 propositions (A, B, C, D).
Une seule bonne réponse par question.

Réponds UNIQUEMENT en JSON valide, sans markdown, sans backticks, sans texte avant
{
  "questions": [
    {
      "question": "texte de la question",
      "options": ["option A", "option B", "option C", "option D"],
      "reponse": 0,
      "explication": "explication courte de la bonne réponse"
    }
  ]
}
La valeur de "reponse" est l'index (0=A, 1=B, 2=C, 3=D).`;

let qcmData;
try {
  const raw = await callGemini(prompt);
  const cleaned = raw.replace(/```json|```/g, '').trim();
  qcmData = JSON.parse(cleaned);
} catch (e) {
  return interaction.followUp({ content: '❌ Erreur lors de la génération du QCM' });
}

const session = {
  questions: qcmData.questions,
  current: 0,
  score: 0,
  oeuvre: oeuvre.nom,
  channelId: interaction.channelId
};
qcmSessions.set(interaction.user.id, session);

await envoyerQuestion(interaction, session, interaction.user.id);
}

```

```

async function envoyerQuestion(interaction, session, userId) {
  const q = session.questions[session.current];
  const embed = new EmbedBuilder()
    .setColor(0x2B2D31)
    .setTitle(`📖 Question ${session.current + 1}/${session.questions.length}`)
    .setDescription(`**${q.question}**`)
    .setFooter({ text: session.oeuvre });

  const row = new ActionRowBuilder().addComponents(
    ['A', 'B', 'C', 'D'].map((letter, i) =>
      new ButtonBuilder()
        .setCustomId(`qcm_rep_${userId}_${i}`)
        .setLabel(`${letter} - ${q.options[i]}`)
        .setStyle(ButtonStyle.Secondary)
    )
  );

  const method = interaction.replied || interaction.deferred ? 'followUp' : 'repl
  await interaction[method]({ embeds: [embed], components: [row], ephemeral: true
}

// ——— COMMANDE /essay —————
async function handleEssay(interaction) {
  await interaction.deferReply();

  const prompt = `Tu es un professeur de Français pour le Bac de Français 2026 en
  Génère une problématique originale pour une dissertation littéraire.
  Elle doit porter sur l'une des œuvres au programme : Cahiers de Douai (Rimbaud),
  La problématique doit être ouverte, littéraire, et adaptée au niveau Terminale.
  Réponds UNIQUEMENT avec la problématique, sans introduction ni commentaire.`;

  const problematique = await callGemini(prompt);

  // Créer un thread
  const thread = await interaction.channel.threads.create({
    name: `📝 Dissertation — ${interaction.user.username}`,
    autoArchiveDuration: 1440,
    reason: 'Session dissertation Bac de Français'
  });

  const embed = new EmbedBuilder()
    .setColor(0x5865F2)
    .setTitle('📝 Ta Dissertation — Bac de Français 2026')
    .setDescription(`**Problématique : **\n\n*${problematique.trim()}*`)
    .addFields(
      { name: 'Comment ça marche', value: 'Rédige ta dissertation ici, message pa
    )

```

```

        .setFooter({ text: 'Bon courage ! 💪' });

await thread.send({ embeds: [embed] });

essaySessions.set(thread.id, {
  userId: interaction.user.id,
  problematique: problematique.trim(),
  messages: [],
  startTime: Date.now()
});

await interaction.editReply(`✅ Ton espace de dissertation a été créé : ${threa
}

// ——— COMMANDE /end —————
async function handleEnd(interaction) {
  const session = essaySessions.get(interaction.channelId);

  if (!session) {
    return interaction.reply({ content: '❌ Aucune session de dissertation active
  }

  if (session.userId !== interaction.user.id) {
    return interaction.reply({ content: '❌ Ce n\'est pas ta dissertation.', ephe
  }

  await interaction.deferReply();

  const dissertationComplète = session.messages.join('\n\n');

  if (!dissertationComplète.trim()) {
    return interaction.editReply('❌ Tu n\'as pas encore écrit de dissertation. F
  }

  const prompt = `Tu es un correcteur expert du Bac de Français 2026 en France.

Problématique : "${session.problematique}"

Dissertation de l'élève :
"""
${dissertationComplète}
"""

Corrige cette dissertation selon les critères officiels du Bac de Français 2026 :
1. Compréhension du sujet et pertinence de la problématique
2. Construction du plan (thèses, antithèses, synthèse)
3. Qualité des arguments et des exemples littéraires

```

4. Expression écrite (syntaxe, vocabulaire, style)

5. Introduction et conclusion

Pour chaque critère :

- Donne une note sur 4 points
- Explique ce qui est réussi
- Explique ce qui peut être amélioré avec des conseils concrets

Termine par :

- Une note globale sur 20
- Un commentaire bienveillant et encourageant
- Les 3 priorités d'amélioration les plus importantes

Réponds en français, de façon structurée et pédagogique.`;

```
const correction = await callGemini(prompt);

const embed = new EmbedBuilder()
  .setColor(0x57F287)
  .setTitle('✅ Correction de ta Dissertation – Bac de Français 2026')
  .setDescription(correction.length > 4096 ? correction.substring(0, 4093) + '...'
  .setFooter({ text: 'Continue à travailler, tu progresses ! 💪' });

await interaction.editReply({ embeds: [embed] });

// Si la correction est trop longue, l'envoyer en plusieurs parties
if (correction.length > 4096) {
  const reste = correction.substring(4093);
  const chunks = reste.match(/.{1,2000}/gs) || [];
  for (const chunk of chunks) {
    await interaction.channel.send(chunk);
  }
}

essaySessions.delete(interaction.channelId);
}

// ——— COMMANDE /close ———
async function handleClose(interaction) {
  if (!interaction.channel.isThread()) {
    return interaction.reply({ content: '❌ Cette commande ne fonctionne que dans'
  }

  await interaction.reply('🔒 Fermeture du thread dans 3 secondes...');
  setTimeout(async () => {
    await interaction.channel.delete().catch(() => {});
  }, 3000);
}
```



```

}

// — ÉVÉNEMENTS —————
client.once('ready', async () => {
  console.log(`✅ Bot connecté : ${client.user.tag}`);
  await registerCommands();
});

client.on('interactionCreate', async interaction => {
  try {
    // Slash commands
    if (interaction.isChatInputCommand()) {
      if (interaction.commandName === 'qcm') return await handleQcm(interaction);
      if (interaction.commandName === 'essay') return await handleEssay(interaction);
      if (interaction.commandName === 'end') return await handleEnd(interaction);
      if (interaction.commandName === 'close') return await handleClose(interaction);
    }

    // Select menu — choix de l'œuvre pour QCM
    if (interaction.isStringSelectMenu() && interaction.customId.startsWith('qcm_')) {
      const userId = interaction.customId.split('_')[2];
      if (interaction.user.id !== userId) return;
      const session = qcmSessions.get(userId);
      if (!session) return;
      await lancerQcm(interaction, interaction.values[0], session.nbQuestions);
    }

    // Boutons — réponses QCM
    if (interaction.isButton() && interaction.customId.startsWith('qcm_rep_')) {
      const parts = interaction.customId.split('_');
      const userId = parts[2];
      const repIndex = parseInt(parts[3]);

      if (interaction.user.id !== userId) {
        return interaction.reply({ content: '❌ Ce n\'est pas ton QCM.', ephemeral: true });
      }

      const session = qcmSessions.get(userId);
      if (!session) return;

      const q = session.questions[session.current];
      const correct = repIndex === q.reponse;
      if (correct) session.score++;

      const embed = new EmbedBuilder()
        .setColor(correct ? 0x57F287 : 0xED4245)
        .setTitle(correct ? '✅ Bonne réponse !' : '❌ Mauvaise réponse');
    }
  }
});

```

```

        .setDescription(`**Bonne réponse : ${['A', 'B', 'C', 'D'][q.reponse]} - $
await interaction.update({ embeds: [embed], components: [] });

session.current++;

if (session.current >= session.questions.length) {
    // QCM terminé
    const scoreEmbed = new EmbedBuilder()
        .setColor(0x5865F2)
        .setTitle('🎓 QCM terminé !')
        .setDescription(`**Score : ${session.score}/${session.questions.length}
        .setFooter({ text: session.oeuvre });

    await interaction.followUp({ embeds: [scoreEmbed], ephemeral: true });
    qcmSessions.delete(userId);
} else {
    await envoyerQuestion(interaction, session, userId);
}
}
} catch (err) {
    console.error('Erreur interaction:', err);
    const reply = interaction.replied || interaction.deferred ? interaction.follo
    await reply.call(interaction, { content: '❌ Une erreur est survenue. Réessai
}
});

// Collecte les messages de dissertation
client.on('messageCreate', async message => {
    if (message.author.bot) return;
    if (!message.channel.isThread()) return;

    const session = essaySessions.get(message.channel.id);
    if (!session) return;
    if (session.userId !== message.author.id) return;
    if (message.content.startsWith('/')) return;

    session.messages.push(message.content);
});

// — DÉMARRAGE —
client.login(DISCORD_TOKEN);

```